BASICS OF JAVA PROGRAMMING
BPLCK1O5C
MODULE 3

1. Explain Class with an Example

A **class** in Java is a blueprint for creating objects. It defines **instance variables** (data) and **methods** (behaviour). The general form of a class is:

General Form of a Class :

```
class ClassName {
    // Instance variables
    type variable1;
    type variable2;

    // Methods
    returnType methodName(parameters) {
        // method body
    }
}
```

Example :

```
class Box {
    double width, height, depth;

    // Constructor
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    // Method to calculate volume
    double volume() {
        return width * height * depth;
    }
}
```

```
public class BoxDemo {
    public static void main(String[] args) {
        Box myBox = new Box(10, 20, 15);
        System.out.println("Volume: " + myBox.volume());
    }
}
```

2. Explain Two Ways of Argument Passing with Example

Java supports two ways of argument passing:

(a) Call by Value (Passing Primitive Data)

Java passes primitive types by value, meaning changes inside the method do not affect the original values.

Example :

```
class Test {
    void modify(int x) {
        x = x * 2;
    }
}

public class CallByValue {
    public static void main(String[] args) {
        int a = 10;
        Test obj = new Test();
        obj.modify(a);
        System.out.println("Value after method call: " + a);
    }
}
```

(b) Call by Reference (Passing Objects)

Objects are passed **by reference**, meaning changes inside the method affect the original object.

Example :

```
class Test {
    int num;
    Test(int num) {
        this.num = num;
    }
```

```
    void modify(Test obj) {
        obj.num *= 2;
    }
}

public class CallByReference {
    public static void main(String[] args) {
        Test obj = new Test(10);
        obj.modify(obj);
        System.out.println("Value after method call: " + obj.num);
    }
}
```

3. What are Constructors? Explain Types with Example

A constructor is a special method that initializes an object when it is created. It has the same name as the class and does not have a return type.

Types of Constructors:

1. **Default Constructor (No Parameters)**

```
class Box {
    double width, height, depth;

    // Default Constructor
    Box() {
        width = height = depth = 1;
    }
}
```

2. **Parameterized Constructor**

```
class Box {
    double width, height, depth;

    // Parameterized Constructor
    Box(double w, double h, double d) {
        width = w;
        height = h;
```

```
        depth = d;
    }
}
```

3. **Copy Constructor (Pass Object as Argument)**

```
class Box {
    double width, height, depth;

    // Copy Constructor
    Box(Box obj) {
        width = obj.width;
        height = obj.height;
        depth = obj.depth;
    }
}
```

4. Constructor Overloading & Method Overloading

Constructor Overloading

Multiple constructors with different parameter lists.

```
class Box {
    double width, height, depth;

    // Constructor with no parameters
    Box() {
        width = height = depth = -1;
    }

    // Constructor with three parameters
    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    // Constructor with one parameter (cube)
    Box(double len) {
        width = height = depth = len;
    }
```

```
}
```

Method Overloading
Multiple methods with the same name but different parameters.

```
class OverloadDemo {
    void test() {
        System.out.println("No parameters");
    }

    void test(int a) {
        System.out.println("a: " + a);
    }

    void test(int a, int b) {
        System.out.println("a and b: " + a + ", " + b);
    }
}

public class OverloadExample {
    public static void main(String[] args) {
        OverloadDemo obj = new OverloadDemo();
        obj.test();
        obj.test(10);
        obj.test(10, 20);
    }
}
```

5. "this" Keyword in Java

The this keyword refers to the current instance of the class. It helps differentiate instance variables from local variables when they have the same name.

Example:

```
class Example {
    int x;

    Example(int x) {
        this.x = x; // Refers to instance variable
    }
```

```
    void show() {
        System.out.println("Value of x: " + this.x);
    }
}

public class Demo {
    public static void main(String[] args) {
        Example obj = new Example(10);
        obj.show();
    }
}
```

6. Garbage Collection & finalize() Method

Garbage Collection: Java automatically removes unused objects to free memory.

finalize () Method: A special method called before an object is destroyed.

Example:

```
class Demo {
    protected void finalize() {
        System.out.println("Object is being garbage collected!");
    }

    public static void main(String[] args) {
        Demo obj = new Demo();
        obj = null;
        System.gc(); // Request Garbage Collection
    }
}
```

7. Java Program for Stack Operations

```
class Stack {
    int stack[] = new int[10];
    int top;

    Stack() { top = -1; }

    void push(int item) {
        if (top == 9) System.out.println("Stack is full");
        else stack[++top] = item;
    }
```

```
    int pop() {
        if (top < 0) {
            System.out.println("Stack is empty");
            return 0;
        } else return stack[top--];
    }
}

public class TestStack {
    public static void main(String args[]) {
        Stack s = new Stack();
        s.push(10);
        System.out.println("Popped: " + s.pop());
    }
}
```

8. Passing Objects as Argument

Objects can be passed to methods just like primitive data types.

Example:

```
class Box {
    double width;

    Box(double w) { width = w; }

    boolean isEqual(Box obj) {
        return this.width == obj.width;
    }
}

public class Test {
    public static void main(String[] args) {
        Box b1 = new Box(10);
        Box b2 = new Box(10);
        System.out.println(b1.isEqual(b2));
    }
}
```

9. Recursion in Java

Recursion is when a method calls itself to solve smaller instances of a problem.

Example (Factorial Calculation):

```
class Factorial {
    int fact(int n) {
        if (n == 0) return 1; // Base case
        return n * fact(n - 1); // Recursive call
    }
}

public class RecursionDemo {
    public static void main(String[] args) {
        Factorial f = new Factorial();
        System.out.println("Factorial of 5: " + f.fact(5));
    }
}
```

SACHIN (1ST SEM)
DATA SCIENCE
ACET